

4. Archivos binarios

Hasta ahora hemos trabajado con ficheros de texto, esto nos ha sido muy conveniente para poder ver los resultados de nuestros programas y porque es muy común trabajar con estos ficheros para generación de informes, logs, archivos de configuración, etc.

En esta lección trabajaremos con ficheros binarios, a estos ficheros también es frecuente llamarlos ficheros de datos, y están formados por secuencias de bytes. Estos ficheros suelen ser más reducidos en tamaño, ya que guardan en ellos la información mínima necesaria, ya que si queremos guardar un entero (int) necesitaremos 4 bytes o para un long 8 bytes, sin necesidad de tener en cuenta una decodificación(formato). Por otra parte, las personas no podremos leer la información de estos ficheros directamente.

Muchas aplicaciones serán más eficientes guardando la información en ficheros binarios, sobre todo, cuando el usuario no necesite visualizar en ningún momento estos archivos de forma directa. Algunos ejemplos de uso de archivos binarios podrían ser: almacenamiento de datos estructurados, persistencia de datos, archivos multimedia o archivos de sistema, entre otros.

En esta lección no vamos a profundizar en los casos de uso de ficheros binarios, tan solo haremos algún ejemplo para entender cómo leer y escribir información en este tipo de ficheros. Nos centraremos en almacenar en nuestros ficheros binarios información de los tipos primitivos del lenguaje Java, como son boolean, char, byte, int, short, long, float y double.

Las operaciones de lectura y escritura son muy similares a las que hemos trabajado con ficheros de texto. Las clases principales involucradas en la lectura y escritura de archivos binarios son `DataInputStream` y `DataOutputStream`.

Estas clases permiten leer y escribir diferentes tipos primitivos del lenguaje con métodos como: `writeBoolean`, `writeInt`, `readBoolean` o `readInt`, entre otros. Consulta la información de la API para saber más sobre sus constructores y métodos.

Para poder leer los archivos de forma correcta debemos saber cómo se han escrito dichos archivos, ya que nosotros, al leer no podremos saber si lo que estamos leyendo era un int o un float en el momento que se almacenó.

Ejercicios

Ejercicio 1.

Crea un archivo binario con la información de 50 Pokémon, de forma aleatoria. Ten en cuenta que la información que debes almacenar es: Número de Pokémon (ten en cuenta que en la actualidad hay 1025 Pokemon, desde 1-Bulbasaur hasta 1025-Pecharunt), el género, y las siguientes estadísticas (hp, ataque, defensa, ataque especial, defensa especial y velocidad).

Teniendo en cuenta que las estadísticas pueden tomar valores del 20 al 260, usa los tipos de datos que minimicen el tamaño del fichero.

Luego lee el contenido del fichero y vuelca los datos a un fichero en formato texto con la información para cada Pokémon como se indica en la figura.

Entre corchetes deben aparecer guiones proporcionalmente al valor de las estadísticas, si el valor fuese 260 tendría los 20 guiones, y si el valor es 20 tendría un guion.

```
*****
* Número: 123 *
* Género: Hembra *
* Estadísticas: *
* HP (054) [----] *
* Ataque (189) [-----] *
* Defensa (085) [-----] *
* Ataque esp. (154) [-----] *
* Defensa esp. (214) [-----] *
* Velocidad (078) [-----] *
*****
```

Ejercicio 2.

Crea un programa que permita generar ficheros binarios con mensajes cifrados. Para cifrar los mensajes usaremos el siguiente algoritmo:

- Generamos números (short) pseudoaleatorios mediante la clase Random y una semilla (sobrecarga del constructor de Random).
- Cada char de nuestra cadena de texto sin cifrar se suma al número aleatorio y se guarda en el fichero con formato int.
- Para decodificar se leen los datos como int y se restan los números aleatorios de la semilla, y se transforma en char.

Crea un programa que pueda codificar mensajes y otro que pueda leerlos, pero solo funcionará si ambos conocen la semilla.